

Navigating PowerShell Remoting Challenges with PowerShell 7 – Michael Waterman



michaelwaterman.nl/2023/08/27/navigating-powershell-remoting-challenges-with-powershell-7

Michael Waterman

August 27, 2023



In the world of system administration and automation, PowerShell has been a trusted companion for managing tasks efficiently across a variety of environments. With the introduction of PowerShell Core 7, the capabilities of PowerShell expanded further by becoming cross-platform, allowing administrators to manage systems regardless of their operating system. Recently, I embarked on a journey to explore the capabilities of PowerShell Core 7's remoting features, but as often happens in the world of technology, I encountered an unexpected challenge.

In my lab setup, I had carefully configured a network of Windows servers, each armed with PowerShell 7. My intention was to seamlessly manage these servers from a Windows 11 client, also equipped with PowerShell 7. The logical choice was to employ the well-known `Enter-PSSession` cmdlet to establish a remote connection. However, this seemingly straightforward process took an unexpected turn. Instead of connecting to the PowerShell 7 instance on the remote machines, I found myself interacting with PowerShell 5.1.

This encounter with version mismatch ignited a curiosity to dive deeper into the intricacies of PowerShell remoting and uncover the methods to explicitly connect to PowerShell 7's WSMAN interface. Join me as I unravel the journey of understanding and overcoming this challenge, shedding light on the nuances of PowerShell remoting with a focus on PowerShell 7.

In this blog post, I will share the insights gained from my exploration.

Problem statement

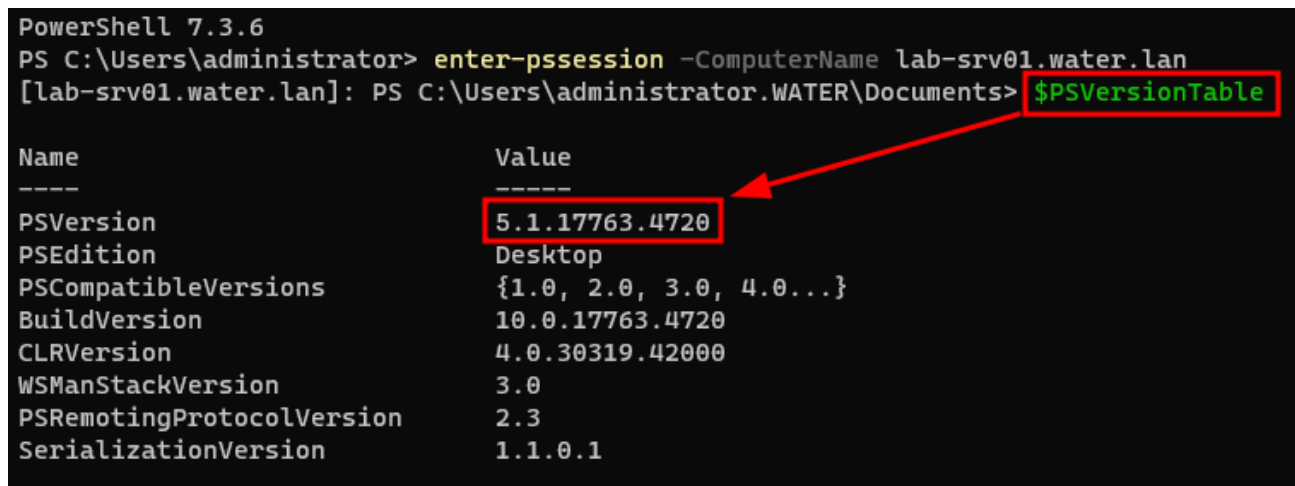
Armed with a lab comprising Windows servers and a Windows 11 client, all equipped with PowerShell 7, my intention was to streamline remote management using the trusty Enter-PSsession cmdlet. Little did I know that this endeavor would lead me to uncover an intriguing version discrepancy.

The initial signs of this enigma appeared innocuously. After executing Enter-PSsession to connect to one of the Windows servers, I promptly fired the \$PSVersionTable cmdlet to examine the PowerShell version of the remote system. Much to my surprise, the version reported was 5.1, not the expected 7 that I had anticipated.

My Windows 11 client and the Windows servers were all equipped with PowerShell 7, and my assumption was that the remote sessions would automatically align with the PowerShell 7 environment. However, reality had other plans, as the remote session seemed to default to PowerShell 5.1.

```
PowerShell 7.3.6
PS C:\Users\administrator> enter-psession -ComputerName lab-srv01.water.lan
[lab-srv01.water.lan]: PS C:\Users\administrator.WATER\Documents> $PSVersionTable
```

Name	Value
PSVersion	5.1.17763.4720
PSEdition	Desktop
PSCompatibleVersions	{1.0, 2.0, 3.0, 4.0...}
BuildVersion	10.0.17763.4720
CLRVersion	4.0.30319.42000
WSManStackVersion	3.0
PSRemotingProtocolVersion	2.3
SerializationVersion	1.1.0.1



Let's find out how this can be fixed!

PowerShell Remoting: A Quick Rundown

PowerShell remoting is a powerful feature that allows administrators and users to manage remote systems by executing commands and scripts on them from a local machine. It enables seamless interaction with remote computers, making it possible to perform administrative tasks, retrieve information, and automate processes across a network without having to physically access each machine.

Key Concepts:

1. **Session:** A PowerShell remoting session is a connection established between a local and a remote computer. This session allows you to send commands and receive output as if you were working directly on the remote machine.

2. **Remote Session Configuration:** On the remote machine, a session configuration (also known as an endpoint) defines which PowerShell commands and features can be used in the remote session. This configuration ensures security and control over what operations can be performed remotely.
3. **Enter-PSSession:** This cmdlet is used to initiate an interactive remote session with a remote computer. It allows you to interact with the remote machine's PowerShell environment as if you were using it directly. You can run commands, navigate the file system, and more.
4. **New-PSSession:** This cmdlet creates a persistent remote session (or session state) that can be reused for multiple commands. This is particularly useful for reducing the overhead of establishing a new session for each command.
5. **Invoke-Command:** This cmdlet lets you execute scripts or commands on a remote computer. You can pass in script blocks or script files, and the results are returned to the local machine.
6. **Exit-PSSession:** This cmdlet is used to end an interactive remote session that was initiated using Enter-PSSession. It returns you to the local machine's session.

Setting Up PowerShell Remoting:

1. **Enable Remoting:** PowerShell remoting must be enabled on both the local and remote machines. On Windows, you can use the Enable-PSRemoting cmdlet to configure the necessary settings.
2. **Firewall Considerations:** Remoting requires specific ports to be open in the firewall. The necessary ports can be configured using Group Policy or by running the Enable-PSRemoting cmdlet with appropriate parameters.

Security:

1. **Authentication:** Remoting supports various authentication methods, including Kerberos and NTLM for Windows machines, and SSH for cross-platform remoting.
2. **Encryption:** By default, remoting sessions are encrypted to protect data transferred between the local and remote machines. Specially in domains it's not necessary to enable TLS encryption on top of encryption by Kerberos.

Use Cases: PowerShell remoting is incredibly versatile and can be used for a variety of tasks, including:

- **Configuration Management:** Remotely configure software, services, and settings on multiple machines simultaneously.
- **Software Deployment:** Install or update software across a network of machines.
- **Troubleshooting:** Remotely investigate and resolve issues on remote computers.
- **Data Collection:** Retrieve information, logs, and system data from remote systems.
- **Automation:** Create and deploy scripts to automate repetitive tasks across multiple machines.

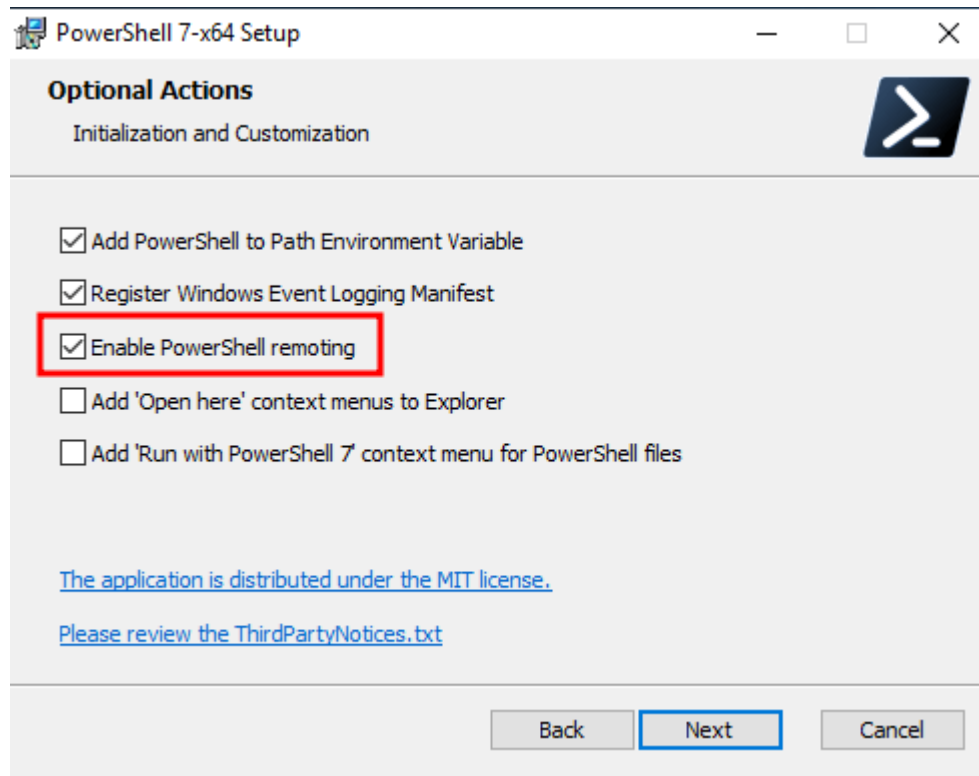
In summary, PowerShell remoting empowers administrators and users to efficiently manage and automate tasks across remote systems. By establishing remote sessions, executing commands, and leveraging various authentication methods, PowerShell remoting enhances productivity and flexibility in managing networked environments.

Installing PowerShell 7: A Step-by-Step Guide

Before you can start using PowerShell 7, you'll need to download and install it. You can obtain the latest version of PowerShell 7 from the official GitHub repository or the PowerShell website. Visit <https://github.com/PowerShell/PowerShell> or <https://learn.microsoft.com/en-us/powershell/scripting/install/installing-powershell-on-windows> to access the downloads.

Installing PowerShell 7 with the GUI:

1. **Download:** Navigate to the official PowerShell website or the GitHub repository to download the latest installer for PowerShell 7.
2. **Run the Installer:** Once the installer executable (e.g., PowerShell-7.x.x-win-x64.msi) is downloaded, run it.
3. **Setup Wizard:** The installer will launch a setup wizard. Follow the on-screen instructions to proceed through the installation process.
4. **Choose Installation Location:** You can either accept the default installation location or choose a different one. Remember the installation path for future reference.
5. **Installation Options:** The wizard may provides options for interacting with PowerShell 7. In our case we should at a minimum select the option "Enable PowerShell remoting".



6. **Completing the Installation:** Once you've made your selections, click "Install" to begin the installation process.
7. **Finish:** Once the installation is complete, you'll see a confirmation message. You can now launch PowerShell 7 from the Start Menu or by typing pwsh in a command prompt.

Note! At the time of writing there seems to be a bug in the GUI installation. Selecting the "Enable PowerShell remoting" option doesn't enable PowerShell remoting. The MSI installation however works as expected. I've reported the bug here:

<https://github.com/PowerShell/PowerShell/issues/20173>

Installing PowerShell 7 Using the MSI File:

1. **Download:** As before, download the latest MSI installer for PowerShell 7 from the official PowerShell website or the GitHub repository.
2. **Command Prompt:** Open a Command Prompt window with administrative privileges.
3. **Navigate to the Download Location:** Use the cd command to navigate to the directory where the downloaded MSI installer is located.
4. **Install:** Run the following command to install PowerShell 7 using the MSI file:

```
msiexec.exe /package PowerShell-7.3.6-win-x64.msi /quiet ENABLE_PSREMOTING=1  
REGISTER_MANIFEST=1 USE_MU=1 ENABLE_MU=1 ADD_PATH=1
```

Replace PowerShell-7.3.6-win-x64.msi with the actual filename of the MSI installer. Just in case I've listed all the options here for reference.

- `ADD_EXPLORER_CONTEXT_MENU_OPENPOWERSHELL` – This property controls the option for adding the Open PowerShell item to the context menu in Windows Explorer.
- `ADD_FILE_CONTEXT_MENU_RUNPOWERSHELL` – This property controls the option for adding the Run with PowerShell item to the context menu in Windows Explorer.
- `ENABLE_PSREMOTING` – This property controls the option for enabling PowerShell remoting during installation.
- `REGISTER_MANIFEST` – This property controls the option for registering the Windows Event Logging manifest.
- `ADD_PATH` – This property controls the option for adding PowerShell to the Windows PATH environment variable.
- `USE_MU` – Use Microsoft Update to update the version of PowerShell.
- `ENABLE_MU` – Enable the use of Microsoft Update.
- `DISABLE_TELEMETRY` – This property controls the option for disabling PowerShell's telemetry by setting the `POWERSHELL_TELEMETRY_OPTOUT` environment variable.

In case you need to enable PowerShell remoting manually, there's actually a script that's provided in the installation folder. Follow these steps to enable the functionality:

- Open a Powershell 7 terminal as administrator.
- `cd` to `$PSHOME`, in my case that's "`C:\Program Files\PowerShell\7`".
- Execute the script, "`Install-PowerShellRemoting.ps1`".
- Restart the WinRM (Windows Remote Management) service, "`Restart-Service winrm`".

Understanding PowerShell Remoting Endpoints

In the realm of PowerShell remoting, an "endpoint" refers to a pre-defined configuration that determines how remote sessions are established and what commands can be executed on a remote computer. Endpoints serve as gateways to the remote machine's PowerShell environment, controlling the scope and capabilities of the remote session.

Key Points:

- **Configuration Control:** Endpoints allow administrators to define specific settings for remote sessions. These settings can include which PowerShell version to use, which modules are available, and what security protocols are required.
- **Security:** Endpoints play a crucial role in ensuring security during remoting. They help enforce security policies by allowing administrators to control which commands and scripts can be executed remotely.
- **Scalability:** By defining endpoints, you can create consistent and tailored remote session experiences for various users and use cases. This scalability enhances the efficiency and manageability of remote administration.

Viewing Endpoint Configuration:

To view the configuration of a remote session endpoint, you can use the `Get-PSSessionConfiguration` cmdlet. This cmdlet provides insights into the available endpoint configurations on a remote computer.

```
PowerShell 7.3.6
PS C:\Users\administrator.WATER> get-PSSessionConfiguration

Name      : PowerShell.7
PSVersion : 7.3
StartupScript :
RunAsUser  :
Permission : NT AUTHORITY\INTERACTIVE AccessAllowed, BUILTIN\Administrators AccessAllowed, BUILTIN\Remote Management Users AccessAllowed

Name      : PowerShell.7.3.6
PSVersion : 7.3
StartupScript :
RunAsUser  :
Permission : NT AUTHORITY\INTERACTIVE AccessAllowed, BUILTIN\Administrators AccessAllowed, BUILTIN\Remote Management Users AccessAllowed
```

As you can see in the picture above, the endpoints have a reference name, stay tuned as this is part of the solution we're looking for.

Connecting to a Specific PowerShell Endpoint

One powerful aspect of PowerShell remoting is the ability to connect to specific endpoints, each tailored to different scenarios and environments. This capability allows us to not only manage version consistency but also customize the remote session experience to match our needs.

Using the `-ConfigurationName` Parameter:

The key to connecting to a particular endpoint lies in the `-ConfigurationName` parameter of the `Enter-PSSession` cmdlet. By specifying the desired endpoint's configuration name, we can establish a remote session that aligns with the version and settings associated with that endpoint.

For instance, let's consider a scenario where we have a Windows server named "*lab-srv01.water.lan*", and we want to connect to a PowerShell 7-specific endpoint configuration named "*powershell.7*". This configuration is tailored to PowerShell 7, ensuring that our remote session operates with PowerShell 7's capabilities and features.

Connecting to PowerShell 7 Endpoint:

```
Enter-PSSession -ComputerName "lab-srv01.water.lan" -ConfigurationName
"powershell.7"
```

In this example, we use the `-ComputerName` parameter to specify the target remote machine (*lab-srv01.water.lan*) and the `-ConfigurationName` parameter to specify the endpoint configuration ("*powershell.7*"). By doing so, we're guaranteed to establish a

remote session that operates under the PowerShell 7 environment, regardless of any other available endpoints.

```
PS C:\Users\administrator> enter-psession -ComputerName lab-srv01.water.lan -ConfigurationName "powershell.7"
[lab-srv01.water.lan]: PS C:\Users\administrator.WATER\Documents> $PSVersionTable
```

Name	Value
PSVersion	7.3.6
PSEdition	Core
GitCommitId	7.3.6
OS	Microsoft Windows 10.0.17763
Platform	Win32NT
PSCompatibleVersions	{1.0, 2.0, 3.0, 4.0...}
PSRemotingProtocolVersion	
SerializationVersion	1.1.0.1
WSManStackVersion	3.0

Connecting a PowerShell 7 Terminal to a PowerShell 5.1 Endpoint

One of the remarkable features of PowerShell remoting is its ability to bridge the gap between different PowerShell versions. While we've explored the importance of version consistency, it's worth noting that a PowerShell 7 terminal can establish a remote connection to a PowerShell 5.1 endpoint.

Cross-Version Compatibility:

PowerShell remoting is designed with a compatibility layer that allows newer versions of PowerShell to communicate with older endpoints. This means that even if you're using a PowerShell 7 terminal, you can still connect to and interact with systems running PowerShell 5.1 through remoting.

Why It Matters:

This cross-version compatibility has practical implications. Imagine you have a mixed environment where some servers are still running PowerShell 5.1 while others have been upgraded to PowerShell 7. Instead of juggling different terminal instances for each version, you can use your PowerShell 7 terminal to connect to any endpoint, regardless of its PowerShell version. This simplifies your administrative tasks and streamlines your workflow.

Obtaining the PowerShell 5.1 Endpoints:

Use the following command-let in a PowerShell 5.1 terminal to obtain the endpoint names:

```
Get-PSSessionConfiguration | where PSVersion -NotContains "7.0" | Select-Object Name, PSVersion
```


Name	PSVersion
----	-----
microsoft.powershell	5.1
microsoft.powershell.workflow	5.1
microsoft.powershell32	5.1
microsoft.windows.servermanagerworkflows	3.0

So, for example we have an open PowerShell 7 terminal and we would like to connect specifically to a certain endpoint, we could use something like this:

```
enter-ssession -ComputerName lab-srv01.water.lan -ConfigurationName
"microsoft.powershell"
```

```
PS C:\Users\administrator> enter-ssession -ComputerName lab-srv01.water.lan -ConfigurationName "microsoft.powershell"
[lab-srv01.water.lan]: PS C:\Users\administrator.WATER\Documents> $PSVersionTable
```

Name	Value
----	-----
PSVersion	5.1.17763.4720
PSEdition	Desktop
PSCompatibleVersions	{1.0, 2.0, 3.0, 4.0...}
BuildVersion	10.0.17763.4720
CLRVersion	4.0.30319.42000
WSManStackVersion	3.0
PSRemotingProtocolVersion	2.3
SerializationVersion	1.1.0.1

Update: 29-08-2023

Just found another bug. After delegating remote access using:

```
set-PSSessionConfiguration -Name "PowerShell.7" -showSecurityDescriptorUI
```

And trying to logon remotely with the delegated account using this command:

```
enter-PSSession -ComputerName "lab-dc01.water.lan" -ConfigurationName
"PowerShell.7"
```

I got a weird error message, that showed me this:

```
Enter-PSSession: Connecting to remote server lab-dc01.water.lan failed with the
following error message : <f:WSManFault
xmlns:f="http://schemas.microsoft.com/wbem/wsman/1/wsmanfault" Code="2689860592"
Machine="lab-dc01.water.lan"><f:Message><f:ProviderFault provider="PowerShell.7"
path="C:\Windows\system32\PowerShell\7.3.6\pwrshplugin.dll"></f:ProviderFault>
</f:Message></f:WSManFault> For more information, see the
about_Remote_Troubleshooting Help topic.
```

Which didn't make sense to me. I fired up [sysinternals ProcMon](#) and discovered that there's a write attempt using the remote users credentials to the file

"RemotePowerShellConfig.txt", that's located in the directory,
"C:\Windows\System32\PowerShell\7.3.6".

Obviously this is not the behavior I would like to see, and I'm assuming there's a programming error. I filed another bug report on GitHub:

<https://github.com/PowerShell/PowerShell/issues/20180>

The workaround for now is to set an ACE with write permissions for the delegated user on the file "RemotePowerShellConfig.txt". Not really a big security issue, but still, this should have been caught during testing.

Conclusion: Navigating PowerShell Remoting with PowerShell Core 7

In the realm of system administration and automation, PowerShell remoting has proven its value in managing remote systems efficiently. Through this exploration of PowerShell remoting with PowerShell Core 7, I've uncovered its capabilities and navigated challenges.

I've began by understanding the role of PowerShell remoting in connecting and managing remote systems. Despite encountering version discrepancies during remote sessions, I learned that using the `-ConfigurationName` parameter with the `Enter-PSSession` cmdlet provides a direct path to desired endpoints, ensuring specific PowerShell versions and tailored settings.

Moreover, I realized that the flexibility of PowerShell remoting extends to cross-version compatibility. A PowerShell 7 terminal effortlessly connects to PowerShell 5.1 endpoints, simplifying management in mixed-environment scenarios.

In conclusion, PowerShell remoting empowers us to manage remote systems with precision and flexibility. By embracing its tools, we're equipped to streamline tasks, troubleshoot issues, and automate processes across diverse environments.

I hope this information is useful and don't hesitate to ask questions or comment on the blog post.